

Chapter 9

References, composition and modules

Object-oriented TypeScript, test first

Aliasing Lab

1. Make basket B from basket A:

```
const b = a;
```

```
const b = [...a];
```

```
const b = structuredClone(a);
```

2. Change basket B:

```
Add eggs to B
```

```
One more of B's first item
```

```
Start again
```

```
const b = [...a];
```

Basket A

- bread × 2
- milk × 2

Basket B

- bread × 2
- milk × 2
- eggs × 6

A and B are different arrays, but share 2 item objects

Today

- copied or shared? values and **references**
- `const` is not "constant"
- three ways to copy - and what each shares
- aliasing bugs, found by tests
- composition and aggregation: objects that **have** objects
- ES modules: `import type`, folders, `index.ts`
- fresh test data, every time

Copied

```
Deno.test("numbers are copied", () => {  
  const a = 10;  
  let b = a;  
  b += 1;  
  assertEquals(a, 10);  
});
```

- primitives: number, string, boolean, null, undefined

Shared

```
Deno.test("arrays are shared", () => {  
  const a = ["bread"];  
  const b = a;  
  b.push("milk");  
  assertEquals(a, ["bread", "milk"]);  
  assertEquals(b, a);  
});
```

- a variable holds a **reference** to an object
- two references to one object: **aliases**
- exactly Java's rule

Same object, or same contents?

```
const a = { name: "milk", quantity: 2 };  
const b = { name: "milk", quantity: 2 };  
assertEquals(a, b);           // same contents  
assertEquals(a === b, false); // different objects
```

Ask	Use
the same object?	<code>===</code> , <code>assertStrictEquals</code>
the same contents?	<code>assertEquals</code>

`const` is not "constant"

```
const shopping = ["bread"];  
shopping.push("milk");    // fine  
shopping = ["cake"];     // error
```

TS2588 [ERROR]: Cannot assign to 'shopping'
because it is a constant.

- `const` fixes the **variable**, not the object - like Java's `final`

Parameters are references too

```
const addMilk = (list: string[]): void => {  
  list.push("milk");  
};  
const shopping = ["bread"];  
addMilk(shopping);  
// shopping is now ["bread", "milk"]
```

- the function gets a **copy of the reference**
- it can change the object, not the caller's variable

The Aliasing Lab

Aliasing Lab

1. Make basket B from basket A:

```
const b = a;
```

```
const b = [...a];
```

```
const b = structuredClone(a);
```

2. Change basket B:

```
Add eggs to B
```

```
One more of B's first item
```

```
Start again
```

```
const b = [...a];
```

Basket A

- bread × 2
- milk × 2

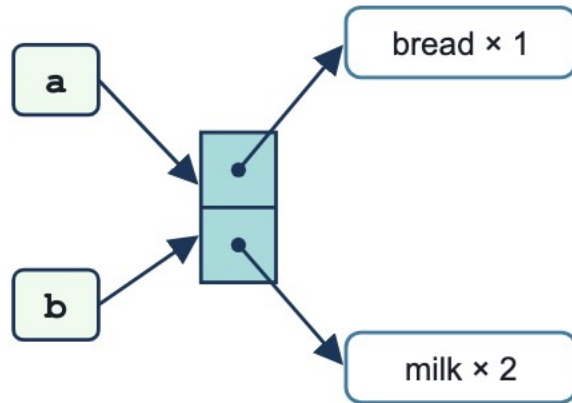
Basket B

- bread × 2
- milk × 2
- eggs × 6

A and B are different arrays, but share 2 item objects

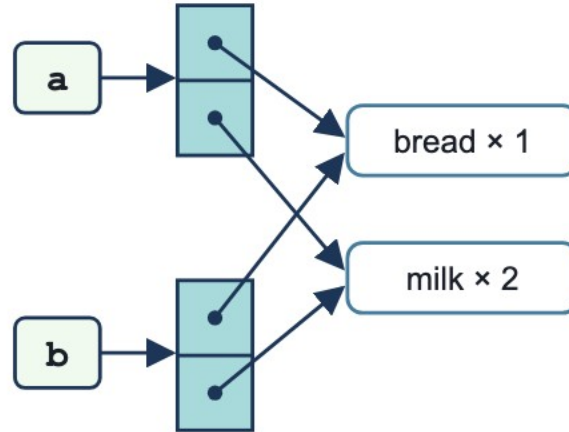
Three ways to make B from A

`const b = a;`



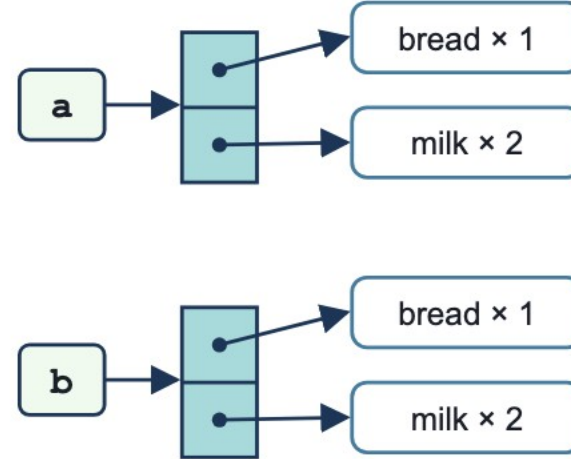
one array, shared

`const b = [...a];`



two arrays, shared items

`const b = structuredClone(a);`



nothing shared

In code

```
switch (mode) {  
  case "same array":  
    return basket;           // no copy  
  case "spread copy":  
    return [...basket];      // shallow  
  case "structuredClone":  
    return structuredClone(basket); // deep  
}
```

- `{ ...item }` - spread for objects, also shallow

Predict, then check

`const b = [...a];` then:

- one more of B's first item - twice
- add eggs to B

What does **A** show?

bread × 3, milk × 2 - items shared, arrays not

Test first: `withItem`

```
Deno.test("withItem leaves the original basket as it was",
  () => {
    const original = makeBasket();
    withItem(original, EGGS);
    assertEquals(original.length, 2);
  });
```

```
// first attempt
basket.push(item);
return basket;
```

Red

not ok 8 - withItem leaves the original basket as it was
[Diff] Actual / Expected

- 3
+ 2

```
export const withItem = (basket: Item[], item: Item)  
  : Item[] => [...basket, item];
```

Green - return something **new**

Careful with `structuredClone`

- copies **data**, not classes
- a cloned `Box` is a plain object: `instanceof Box` is `false`
- its methods are gone - and TypeScript still says `Box`
- plain data: fine. Your classes: write a copy method

A playlist has songs

Playlist

Library

1. Harbour Lights - The Tidewaters (3:34) +
2. Paper Planes at Noon - Mira Quinn (3:07) +
3. Slow Train North - Old Atlas (4:23) +
4. Neon Rain - Kilowatt Kids (3:18) +
5. Lantern - Mira Quinn (4:01) +
6. Copper Sky - The Tidewaters (5:05) +
7. Half a Melody - June & the Jetsam (2:52) +
8. Static Hearts - Kilowatt Kids (3:49) +

My playlist

4 songs · 15:05

1. Harbour Lights - The Tidewaters (3:34) ↑ ×
2. Slow Train North - Old Atlas (4:23) ↑ ×
3. Paper Planes at Noon - Mira Quinn (3:07) ↑ ×
4. Lantern - Mira Quinn (4:01) ↑ ×

- **composition:** the whole owns and makes its parts
- **aggregation:** it refers to parts that live on their own

Sharing is safe if nothing can change

```
export class Song {  
  constructor(  
    public readonly title: string,  
    public readonly artist: string,  
    public readonly seconds: number,  
  ) { ... }  
}
```

- library and playlists share the **same** Song objects
- **readonly** : no alias can surprise you

The leaking getter

```
public getSongs(): Song[] {  
    return this.songs;    // the private array itself!  
}
```

not ok 11 – changing the array from getSongs does not
change the playlist

– 4
+ 3

`private` protects the field, not the array it refers to

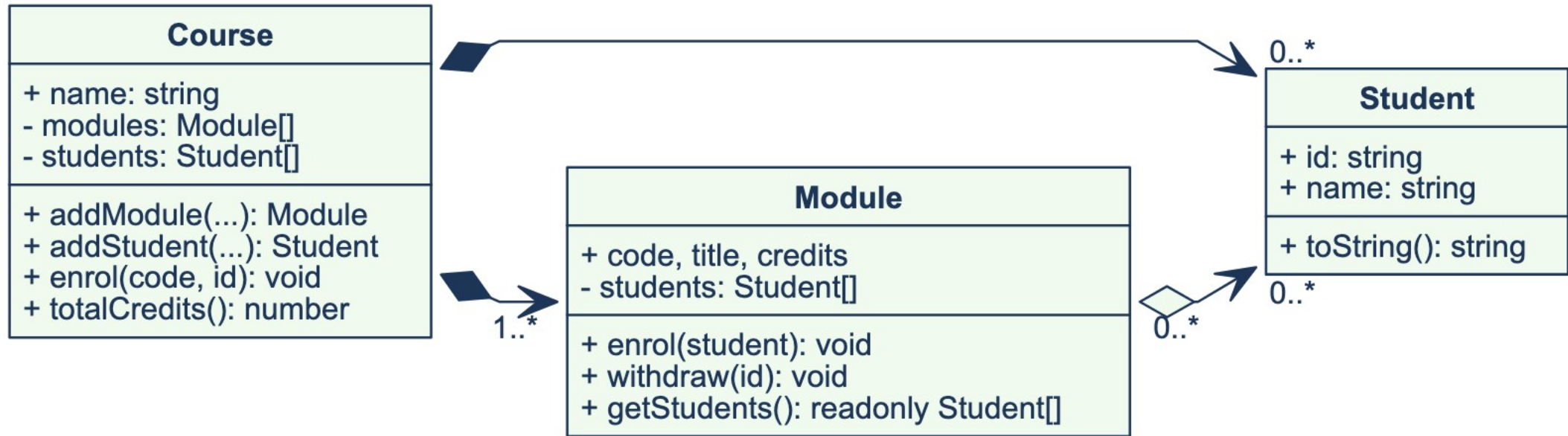
Copy, and say so in the type

```
public getSongs(): readonly Song[] {  
    return [...this.songs];  
}
```

```
TS2339 [ERROR]: Property 'push' does not exist on  
type 'readonly Song[]'.
```

- `readonly` - compile time only; the copy is the real protection
- copy coming in too: `this.songs = [...songs];`

Course has Modules has Students



Filled diamond: composition - the course creates its modules and students.

Hollow diamond: aggregation - a module shares the course's Student objects.

Identity or id?

```
public isEnrolled(student: Student): boolean {  
    return this.students.some(  
        (enrolled) => enrolled.id === student.id,  
    );  
}
```

- playlist: "this very song?" - `===`
- module: "this person?" - compare ids
- a design decision

ES modules

- every file with `import` / `export` is a module
- not exported = invisible everywhere else
- **named exports** only in this book

```
export class Course { ... }  
import { courseSummary } from "../report.ts";
```

- relative imports end in `.ts`

import type

```
import type { Course, Module } from "../model/index.ts";  
import { addItem, type Item } from "../basket.ts";
```

- types only - gone after compiling
- use it as a value and:

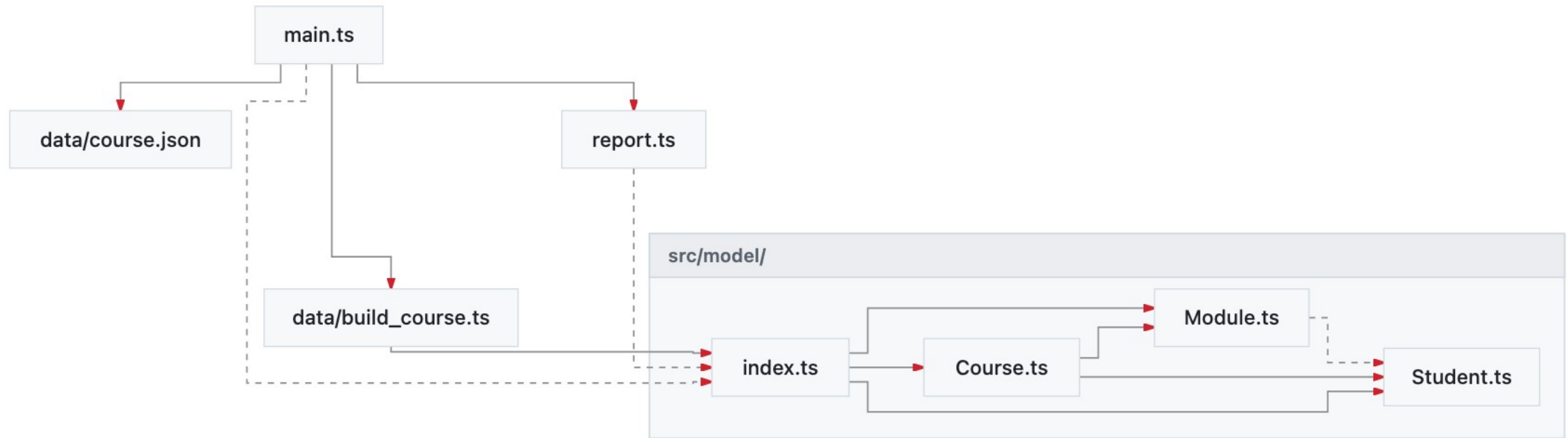
```
TS1361 [ERROR]: 'Course' cannot be used as a value  
because it was imported using 'import type'.
```

A folder with a front door

```
// src/model/index.ts  
export { Course } from "./Course.ts";  
export { Module } from "./Module.ts";  
export { Student } from "./Student.ts";
```

```
import { Course } from "../model/index.ts";
```

Who imports whom



Solid: `import { ... }`. Dashed: `import type { ... }` - gone after compiling.

Dependencies go one way - never in a circle

Java packages and ES modules

Java	TypeScript
package = folder, <code>package model;</code>	module = file
<code>public class</code>	<code>export class</code>
package-private	not exported (file-private)
<code>import model.Course;</code>	<code>import { Course } from "..."</code>
-	<code>index.ts</code> as a front door

A shared fixture

```
const COURSE = makeCourse();

Deno.test("enrol adds Bob to Maths", () => {
  COURSE.enrol("MATH", "S2");
  assertEquals(moduleOf(COURSE, "MATH").count(), 1);
});
Deno.test("nobody is on Maths yet", () => {
  assertEquals(moduleOf(COURSE, "MATH").count(), 0);
});
```

The second test fails (actual 1, expected 0) – yet passes on its own!

Fresh data, every time

```
export const makeCourse = (): Course => {  
  const course = new Course("Test course");  
  course.addStudent("S1", "Ann");  
  // ...  
  return course;  
};
```

- each test: `const course = makeCourse();`
- Java: `@BeforeEach` - here, a plain function

Summary

- primitives copied; objects shared - beware aliases
- `[...a]` shallow, `structuredClone` deep (plain data)
- return something new; test the original is unchanged
- copy arrays into and out of a class; `readonly T[]`
- composition owns; aggregation shares
- named exports, `import type`, `index.ts`
- fixtures are functions

Challenges

1. `withoutItem` - a new basket
2. `moveDown`
3. duplicate a playlist
4. which modules is a student on?
5. a second folder: `report/` and CSV
6. save and restore with `toData()`

All test first